

## 2교시: Dockerfile 기본 문법

### 수업 목표

- image/build/registry 개념을 실행 evidence로 확인한다.
- 명령, 검증, cleanup을 분리해 기록한다.
- 실패를 RCA 형식으로 정리한다.

### 강의 전개

FROM/WORKDIR/COPY/RUN/CMD/EXPOSE를 실행 계약으로 읽는다.

이 교시는 설명만 듣고 지나가지 않는다. 명령은 반드시 code block으로 실행하고, 바로 이어서 검증 명령을 실행한다. 정상 출력이 다를 수 있는 부분은 전체 문자열을 외우지 않고 성공 패턴을 기록한다. 실패도 수업 산출물이다. 실패한 명령, 에러 요약, 가설, 다시 확인한 명령을 함께 남긴다.

### 실습 명령

```
mkdir -p week2/day3/labs/static-site
printf "<h1>day3 static app</h1>" > week2/day3/labs/static-site/index.html
printf "FROM nginx:1.27-alpine\nCOPY index.html\n/usr/share/nginx/html/index.html\nEXPOSE 80\n" > week2/day3/labs/static-site/Dockerfile
```

### 검증 명령

```
sed -n "1,80p" week2/day3/labs/static-site/Dockerfile
```

### 실패 드릴과 오해 교정

| 상황             | 해석                                                 |
|----------------|----------------------------------------------------|
| build 실패       | Dockerfile path, build context, COPY source를 확인한다. |
| run 성공 후 접속 실패 | EXPOSE와 host -p mapping을 구분한다.                     |
| push 요구        | credential과 public repository gate를 먼저 확인한다.       |

## Cleanup

```
docker stop paperclip-day3-static || true
docker rm paperclip-day3-static || true
# 필요할 때만 실습 image 삭제
# docker image rm paperclip-static-site:day3 paperclip-static-site:day3-reviewed
```

Cleanup은 비용과 데이터 안전을 동시에 다룬다. container를 지우는 명령과 volume/network/image를 지우는 명령은 의미가 다르다. 특히 volume 삭제는 database data 삭제일 수 있으므로 실습 volume인지 확인한 뒤 실행한다.

## Evidence

| 항목               | 제출 기준                    |
|------------------|--------------------------|
| Command evidence | 실행한 build/run/inspect 명령 |
| Verification     | HTTP/history/inspect 결과  |
| RCA              | 실패 drill 원인과 재검증         |

## 강의자 설명 포인트

Day 3의 중심은 "내가 실행한 것은 어떤 artifact인가"라는 질문이다. Day 1과 Day 2에서는 이미 존재하는 official image를 가져와 실행했다. Day 3에서는 source file과 Dockerfile을 image로 포장한다. 학생은 Dockerfile을 단순 설치 스크립트처럼 읽기 쉽지만, 실제로는 build context를 입력으로 받아 image layer를 만드는 build recipe다.

COPY 한 줄은 작아 보이지만 운영적으로는 중요하다. 어떤 파일이 image 안에 들어가고 어떤 파일이 제외되는지에 따라 image size, secret risk, rebuild cache가 달라진다. .dockerignore는 예쁘게 정리하는 파일이 아니라 Docker daemon으로 보내는 build input boundary를 줄이는 장치다. 이 점을 반복해서 강조한다.

## 운영 해석

tag는 이름표고 digest는 content identity에 가깝다. latest는 편하지만 재현성에는 약하다. 같은 tag가 시간이 지나 다른 내용을 가리킬 수 있기 때문이다. 교육 단계에서는 tag를 쉽게 쓰되, 운영 판단에서는 명시적 version tag와 digest 확인이 왜 필요한지 연결한다.

Docker Hub push는 학습자가 하고 싶어할 수 있지만 기본 요구로 두지 않는다. public repository에 secret이나 불필요한 파일이 들어간 image를 올리는 사고를 막아야 한다. push 전에 image 안에 무엇이 들어갔는지, tag가 무엇인지, 공개 범위가 무엇인지 확인하게 한다.

## README 기록 예시

---

```
## Image Build Evidence
- Dockerfile path:
- Build command:
- Image tag:
- Base image:
- Build context note:
- .dockerignore excludes:
- HTTP check:
- history/inspect summary:
- Failure drill:
```

## 다음 연결

---

Day 4는 이 image를 여러 runtime config와 failure 조건으로 실행해 관찰한다.